

Lista 3 - Aprendizado de Máquina

Prof. Eduardo Laber

Alessandra Malizia (2512127) e Luiz Felipe Rodrigues (2511879)

Dezembro de 2025

Questão 1

(a)

Ordenando os pontos de dados em ordem crescente:

$$x^{(1)} \leq x^{(2)} \leq \dots \leq x^{(N)}$$

queremos provar que, em uma solução ótima do K-means unidimensional, cada cluster é formado por pontos consecutivos nessa ordenação.

Prova (por contradição)

Suponha que exista uma atribuição ótima em que um cluster contenha pontos não consecutivos, ou seja,

existem índices $i < j < l$ tais que $x^{(i)}$ e $x^{(l)}$ pertencem ao mesmo cluster C_r , mas $x^{(j)}$ pertence a outro cluster C_s .

Os centroides de cada cluster são as médias dos pontos atribuídos a eles:

$$\mu_r = \frac{1}{|C_r|} \sum_{x \in C_r} x, \quad \mu_s = \frac{1}{|C_s|} \sum_{x \in C_s} x.$$

Como $x^{(i)} < x^{(j)} < x^{(l)}$, temos $\mu_r < \mu_s$.

Portanto, a distância de $x^{(j)}$ ao centro μ_s é maior que a distância ao centro μ_r , violando a atribuição ótima (pois cada ponto deve ser atribuído ao centro mais próximo).

Assim, qualquer solução ótima deve ter a propriedade de que cada cluster contém intervalos consecutivos de pontos.

(b)

Algoritmo de programação dinâmica de complexidade $O(KN^2)$
para o problema unidimensional.

Seja:

$$\text{cost}(p, q) = \sum_{i=p}^q (x^{(i)} - \bar{x}_{p:q})^2,$$

onde $\bar{x}_{p:q}$ é a média dos pontos entre $x^{(p)}$ e $x^{(q)}$.

Essa função representa o erro quadrático total de agrupar os pontos $x^{(p)}, \dots, x^{(q)}$ em um cluster.

Para fazer a recorrência do problema de programação dinâmica, definimos:

$DP[k][n]$ = custo mínimo para particionar os primeiros n pontos em k clusters.

A relação de recorrência é:

$$DP[k][n] = \min_{m < n} \{DP[k-1][m] + \text{cost}(m+1, n)\},$$

onde $DP[1][n] = \text{cost}(1, n)$ (todos os pontos em um único cluster), e o resultado final é $DP[K][N]$.

Podemos calcular $\text{cost}(p, q)$ em tempo $O(1)$ com as somas:

$$S_1(i) = \sum_{t=1}^i x^{(t)}, \quad S_2(i) = \sum_{t=1}^i (x^{(t)})^2.$$

Assim:

$$\bar{x}_{p:q} = \frac{S_1(q) - S_1(p-1)}{q - p + 1},$$

e

$$\text{cost}(p, q) = S_2(q) - S_2(p-1) - \frac{(S_1(q) - S_1(p-1))^2}{q - p + 1}.$$

Dessa forma, o algoritmo completo é:

1. Ordene $x^{(1)}, \dots, x^{(N)}$.
2. Compute S_1 e S_2 .
3. Inicialize $DP[1][n] = \text{cost}(1, n)$.
4. Para $k = 2, \dots, K$:
 - Para $n = k, \dots, N$:

$$- DP[k][n] = \min_{m=k-1, \dots, n-1} [DP[k-1][m] + \text{cost}(m+1, n)].$$

O cálculo de $\text{cost}(p, q)$ é $O(1)$.

A dupla iteração em k e n com varredura de m gera custo total $O(KN^2)$.

Questão 2

(a)

O método EM está maximizando a verossimilhança dos dados X , dado uma mistura de duas distribuições gaussianas com média, variância e probabilidade da componente desconhecidos.

$$p(X | \pi, \mu, \sigma^2) = \prod_{i=1}^n \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \sigma_k^2).$$

Como maximizar diretamente essa verossimilhança é um problema complicado (não convexo), podemos obter as estimativas iterativamente usando o algoritmo EM.

O EM alterna entre:

- E-step (Expectation step), que computa as responsabilidades:

$$\gamma_{ik} = p(z_i = k | x_i) = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \sigma_k^2)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \sigma_j^2)}.$$

- e M-step, que maximiza o valor esperado do log da verossimilhança e atualiza os pesos, médias e variâncias

O algoritmo aumenta a verossimilhança a cada iteração até convergir.

(b) Passo E

Temos os dados:

$$X = \{1.0, 1.2, 0.8, 4.2, 3.9, 4.4\}.$$

Parâmetros iniciais:

$$\pi_1^{(0)} = \pi_2^{(0)} = 0.5, \quad \mu_1^{(0)} = 1.0, \quad \mu_2^{(0)} = 4.0,$$

$$(\sigma_1^2)^{(0)} = (\sigma_2^2)^{(0)} = 1.0.$$

Cada responsabilidade é:

$$\gamma_{i1} = \frac{\pi_1 \mathcal{N}(x_i | 1, 1)}{\pi_1 \mathcal{N}(x_i | 1, 1) + \pi_2 \mathcal{N}(x_i | 4, 1)}, \quad \gamma_{i2} = 1 - \gamma_{i1}.$$

Como $\pi_1 = \pi_2 = 1/2$, temos:

$$\gamma_{i1} = \frac{\mathcal{N}(x_i | 1, 1)}{\mathcal{N}(x_i | 1, 1) + \mathcal{N}(x_i | 4, 1)}.$$

A densidade Gaussiana unidimensional é:

$$\mathcal{N}(x | \mu, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2}\right).$$

Desconsiderando o denominador comum, temos:

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i-1)^2}{2}\right)}{\exp\left(-\frac{(x_i-1)^2}{2}\right) + \exp\left(-\frac{(x_i-4)^2}{2}\right)}.$$

Calculando para cada ponto:

- Para $x_1 = 1.0$:

$$\gamma_{11} = \frac{e^{-(0)^2/2}}{e^{-(0)^2/2} + e^{-(3)^2/2}} = \frac{1}{1 + e^{-9/2}} \approx 0.9889.$$

- Para $x_2 = 1.2$:

$$\gamma_{21} = \frac{e^{-(0.2)^2/2}}{e^{-(0.2)^2/2} + e^{-(2.8)^2/2}} \approx \frac{e^{-0.02}}{e^{-0.02} + e^{-3.92}} \approx 0.9805.$$

- Para $x_3 = 0.8$:

$$\gamma_{31} \approx \frac{e^{-0.02}}{e^{-0.02} + e^{-5.12}} \approx 0.9898.$$

- Para $x_4 = 4.2$:

$$\gamma_{41} \approx \frac{e^{-5.12}}{e^{-5.12} + e^{-0.02}} \approx 0.0102.$$

- Para $x_5 = 3.9$:

$$\gamma_{51} \approx \frac{e^{-4.205}}{e^{-4.205} + e^{-0.005}} \approx 0.0148.$$

- Para $x_6 = 4.4$:

$$\gamma_{61} \approx \frac{e^{-5.76}}{e^{-5.76} + e^{-0.08}} \approx 0.0067.$$

Logo,

x_i	γ_{i1}	γ_{i2}
1.0	0.9889	0.0111
1.2	0.9805	0.0195
0.8	0.9898	0.0102
4.2	0.0102	0.9898
3.9	0.0148	0.9852
4.4	0.0067	0.9933

(c) Passo M

O passo M atualiza:

$$N_k = \sum_{i=1}^n \gamma_{ik}, \quad \pi_k = \frac{N_k}{n},$$

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} x_i, \quad \sigma_k^2 = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} (x_i - \mu_k)^2.$$

Cálculo de N_1 e N_2

$$N_1 = 0.9889 + 0.9805 + 0.9898 + 0.0102 + 0.0148 + 0.0067 \approx 2.991.$$

$$N_2 = 6 - N_1 = 3.009.$$

Logo:

$$\pi_1 = \frac{N_1}{6} \approx 0.4985, \quad \pi_2 = \frac{N_2}{6} \approx 0.5015.$$

Cálculo da nova média μ_1

$$\mu_1 = \frac{1}{N_1} (0.9889 \cdot 1.0 + 0.9805 \cdot 1.2 + 0.9898 \cdot 0.8 + 0.0102 \cdot 4.2 + 0.0148 \cdot 3.9 + 0.0067 \cdot 4.4).$$

Computando:

- $0.9889 \cdot 1.0 \approx 0.9889$
- $0.9805 \cdot 1.2 \approx 1.1766$
- $0.9898 \cdot 0.8 \approx 0.7918$
- $0.0102 \cdot 4.2 \approx 0.0428$
- $0.0148 \cdot 3.9 \approx 0.0577$
- $0.0067 \cdot 4.4 \approx 0.0295$

Somando:

$$\sum \gamma_{i1} x_i \approx 3.0873.$$

Logo:

$$\mu_1 \approx \frac{3.0873}{2.991} \approx 1.032.$$

Cálculo da nova média μ_2

Computando:

$$\mu_2 \approx \frac{0.0111 \cdot 1.0 + 0.0195 \cdot 1.2 + 0.0102 \cdot 0.8 + 0.9898 \cdot 4.2 + 0.9852 \cdot 3.9 + 0.9933 \cdot 4.4}{3.009}.$$

Numerador aproximadamente:

$$0.0111 + 0.0234 + 0.0081 + 4.157 + 3.848 + 4.370 \approx 12.417.$$

Então:

$$\mu_2 \approx \frac{12.417}{3.009} \approx 4.126.$$

Cálculo das variâncias

A variância do cluster 1:

$$\sigma_1^2 = \frac{1}{N_1} \sum_i \gamma_{i1} (x_i - \mu_1)^2 \approx \frac{1}{2.991} (0.0029 + 0.0284 + 0.0540 + \dots) \approx 0.04.$$

A variância do cluster 2:

$$\sigma_2^2 \approx 0.03.$$

Assim,

$\pi_1 \approx 0.4985, \quad \mu_1 \approx 1.032, \quad \sigma_1^2 \approx 0.04,$
$\pi_2 \approx 0.5015, \quad \mu_2 \approx 4.126, \quad \sigma_2^2 \approx 0.03.$

(d)

Os três primeiros pontos $\{1.0, 1.2, 0.8\}$ têm responsabilidades muito próximas de 1 para o cluster 1. Já os três últimos pontos $\{4.2, 3.9, 4.4\}$, têm responsabilidades muito próximas de 1 para o cluster 2.

A mistura Gaussiana maximiza a verossimilhança atribuindo alta probabilidade a pontos perto de cada média. Em regiões onde a Gaussiana 1 domina, $\gamma_{i1} \approx 1$ e $\gamma_{i2} \approx 0$. Em regiões onde a Gaussiana 2 domina, ocorre o oposto.

Mesmo sendo um método probabilístico, as responsabilidades no algoritmo EM convergem rapidamente para valores próximos de 0 ou 1 quando os clusters estão bem separados, o que leva a um comportamento de clustering.

(e)

O EM sempre aumenta (ou mantém) a verossimilhança por construção, porque:

- E-step: Calcula o valor esperado do log da verossimilhança completa usando a distribuição posterior atual.
Isso sempre cria um *bound* inferior da verossimilhança.
- M-step: maximiza esse bound em relação aos parâmetros.
Isso não pode diminuir o bound, apenas aumentar ou manter.

Como a verossimilhança observada é sempre maior ou igual ao bound, o resultado é:

$$\mathcal{L}(w^{(t+1)}) \geq \mathcal{L}(w^{(t)}).$$

Portanto, o EM não reduz a verossimilhança a cada iteração.

Questão 3

Considere o modelo de regressão linear:

$$y^{(i)} = w^\top x^{(i)} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2),$$

e, em forma matricial:

$$y = Xw + \epsilon, \quad y|X, w \sim \mathcal{N}(Xw, \sigma^2 I).$$

Além disso, o vetor de parâmetros possui um prior Gaussiano:

$$w \sim \mathcal{N}\left(0, \frac{2\sigma^2}{\lambda} I\right), \quad \lambda > 0.$$

Queremos mostrar que o estimador MAP de w é o mesmo que o estimador da **regressão ridge**, cuja solução é:

$$w* = \arg \min_w \|y - Xw\|_2^2 + \lambda \|w\|_2^2.$$

Solução

A densidade posterior para a estimação por MAP é:

$$p(w|X, y) \propto p(y|X, w) p(w),$$

onde podemos calcular a verossimilhança e a prior como:

Verossimilhança:

$$p(y|X, w) \propto \exp\left(-\frac{1}{2\sigma^2}\|y - Xw\|_2^2\right)$$

Prior Gaussiano:

$$p(w) \propto \exp\left(-\frac{1}{2}w^\top \left(\frac{\lambda}{2\sigma^2}I\right) w\right) = \exp\left(-\frac{\lambda}{4\sigma^2}\|w\|_2^2\right)$$

Tomando o logaritmo e ignorando constantes independentes de w :

$$-\log p(w|X, y) = \frac{1}{2\sigma^2}\|y - Xw\|_2^2 + \frac{\lambda}{4\sigma^2}\|w\|_2^2 + \text{cte.}$$

Encontramos o estimador MAP minimizando o negativo do logaritmo da posterior. Multiplicando tudo por $2\sigma^2$ (não afeta o minimizador), obtemos:

$$\arg \min_w \left[\|y - Xw\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \right].$$

Podemos redefinir o hiperparâmetro da penalização como $\lambda_{\text{ridge}} = \lambda/2$ obtendo o mesmo resultado da regressão ridge:

$$w_{\text{MAP}} = \arg \min_w \left(\|y - Xw\|_2^2 + \lambda_{\text{ridge}}\|w\|_2^2 \right).$$

Logo,

$w_{\text{MAP}} = w_{\text{ridge}}$

Questão 4

Considere um dataset com n pontos em $\mathbb{R}^{1000}$, armazenados em uma matriz:

$$A \in \mathbb{R}^{n \times 1000},$$

cada linha sendo um ponto. Sabemos que $\text{rank}(A) = 10$.

Recebemos um *stream* de pontos $X = \{x^1, \dots, x^m\}$ e queremos, para cada x^i , encontrar os **5 pontos mais próximos** no dataset.

(a) Produto interno

A similaridade entre dois pontos é dada pelo produto interno:

$$\text{sim}(x, a_j) = x^\top a_j,$$

e queremos os pontos mais próximos.

Como a matriz A possui posto 10, podemos aplicar a decomposição SVD e representá-la apenas com os 10 primeiros valores singulares (que são os únicos não nulos):

$$A = U_{10} \Sigma_{10} V_{10}^\top,$$

onde:

- $U_{10} \in \mathbb{R}^{n \times 10}$
- $\Sigma_{10} \in \mathbb{R}^{10 \times 10}$
- $V_{10} \in \mathbb{R}^{1000 \times 10}$.

Cada linha da matriz pode ser reescrita no espaço reduzido de 10 dimensões:

$$a_j^T = V_{10} \Sigma_{10} (U_{10}^\top)_j, \quad j = 1, \dots, n$$

onde $(U_{10}^\top)_j$ é a j -ésima linha de $U_{10}^\top$.

Passando cada linha para o espaço reduzido, obtemos:

$$a_j = z_j V_{10}^T, \quad \text{onde} \quad z_j = \Sigma_{10} (U_{10}^\top)_j, \quad j = 1, \dots, n.$$

Da mesma forma, projetamos cada ponto do *stream*:

$$x_{\text{proj}} = \Sigma_{10}^{-1} V_{10}^\top x.$$

Como:

$$x^\top a_j = x_{\text{proj}}^\top z_j,$$

podemos computar todos os produtos internos no espaço de 10 dimensões, reduzindo o custo de $O(1000n)$ para $O(10n)$. Assim, basta selecionar os 5 maiores valores.

(b) Distância Euclidiana

Neste caso, a similaridade entre dois pontos é dada pela distância Euclidiana:

$$\text{sim}(x, a_j) = \|x - a_j\|_2$$

Usamos novamente a decomposição SVD:

$$A = U_{10}\Sigma_{10}V_{10}^\top.$$

Cada linha de A é projetada no espaço de dimensão reduzida como:

$$z_j = \Sigma_{10}(U_{10}^\top)_j,$$

e cada ponto novo:

$$x_{\text{proj}} = \Sigma_{10}^{-1}V_{10}^\top x.$$

Como a distância euclidiana é preservada no espaço reduzido, vale:

$$\|x - a_j\|_2^2 = \|x_{\text{proj}} - z_j\|_2^2$$

Assim, podemos realizar a busca de vizinhos no espaço reduzido de dimensão 10:

$$j^* = \arg \min_j \|x_{\text{proj}} - z_j\|_2.$$

Dessa forma, o custo no cálculo das distâncias também cai de $O(1000n)$ para $O(10n)$.